



MedBridge AI

Application Médicale avec traduction automatique entre langues à faibles ressources

Hamad ALDENAWI

Christ KELI

Patrice SEBASTIANO

Steve SANOGO

APS Projet 6 | Université d'Avignon - CERI | Avril 2026

Réduire les barrières linguistiques en contexte médical

Problématique :

- Difficulté de communication entre patients et médecins ne partageant pas la même langue
- Problème accentué pour les langues à faibles ressources (ex : Éwé, dialectes arabes)
- Les systèmes actuels sont limités :
 - Sensibles aux variations orthographiques
 - Peu adaptés au domaine médical
- Les métriques classiques (WER) peuvent être trompeuses

→ *Les métriques ne reflètent pas toujours la compréhension réelle.*

MedBridge AI : un assistant vocal médical multilingue

- **Objectif** : faciliter la communication patient–médecin
- **Composants du système** :
 - Reconnaissance vocale (ASR)
 - Identification de la langue (LID)
 - Traduction (MT) : Éwé → Français / Anglais, Arabe → Français / Anglais
 - Synthèse vocale (TTS)
- **Focus** :
 - Langues à faibles ressources
 - Conditions réelles (bruit, variabilité)

Architecture du système



Pipeline complet : entrée vocale → transcription → identification de la langue → traduction → synthèse vocale



Contribution ASR low-resource

OmniASR CTC 300M sur l'Ewe

Steve Sanogo | Encadrement : Yannick Estève
Avignon Université · Master 1 APS · Mars–Avril 2026

OmniASR CTC 300M

Ewe (ewe_Latn)

150h audio

WER norm. 33.35%

L'EWE EN QUELQUES MOTS



6,7 millions
de locuteurs



Parlé dans trois pays
d'Afrique
Subsaharienne



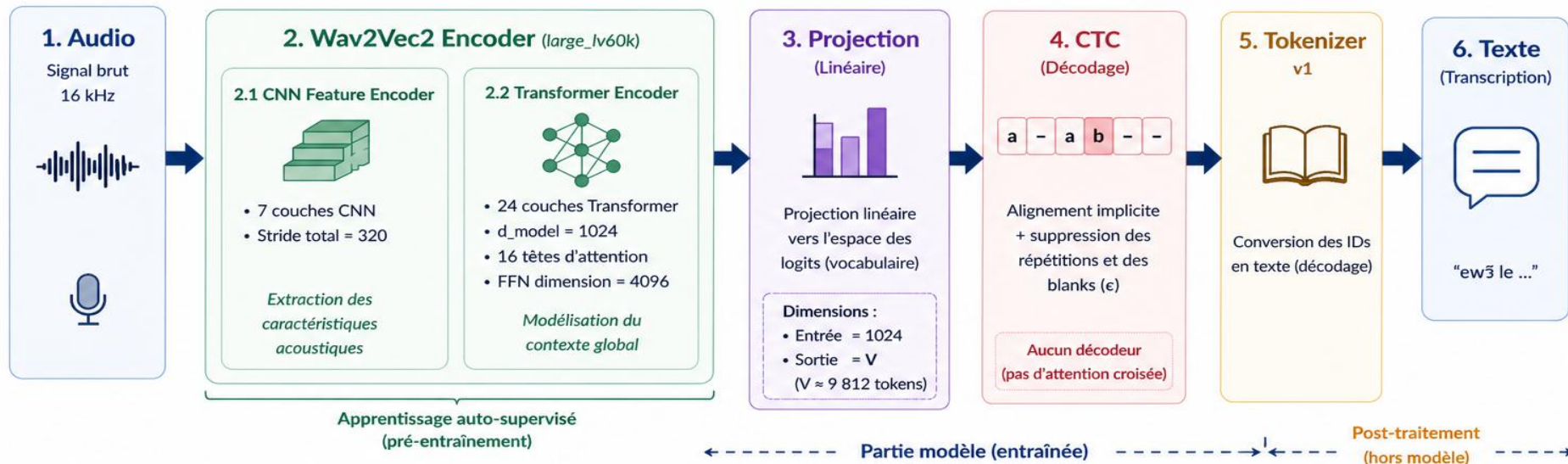
Langue Kwa
de la famille Niger-Congo



Objectif : améliorer la reconnaissance automatique de la parole pour une langue low-resource par un fine-tuning ciblé sur l'Ewe avec des données réalistes.

Architecture OmniASR CTC/300M – Détails & Dimensions

Pipeline complet de la reconnaissance automatique de la parole



DÉTAIL DES COMPOSANTS	
Wav2Vec2 Encoder (large_lv60k)	
CNN Feature Encoder	7 couches 1D Convolution
Stride total	320
Transformer Encoder	
Nombre de couches	24
Dimension (d_{model})	1024
Nombre de têtes d'attention	16
FFN dimension	4096
Dropout	0.1

Projection Linéaire (CTC Head)	
Type	Linear
Entrée	1024
Sortie	Vocabulaire ($V \approx 9\,812$)
Activation	Softmax (dans le CTC)
Paramètres	≈ 10.0 M

CTC	
Fonction de perte	CTC Loss
Blank symbol	ϵ (id = 0)
Décodage	Greedy / Beam Search
Alignement	Implicite (pas d'alignement temporel explicite)

Tokenizer v1	
Type	SentencePiece (BPE)
Vocabulaire	$\approx 9\,812$ tokens
Cas	Sensible à la casse
Langues	Multilingue (99+)

POINTS CLÉS	
✓	Le modèle est un encodeur pur (pas de décodeur Transformer).
✓	CTC permet la transcription sans alignement temporel explicite.
✓	La projection linéaire mappe les représentations (1024) vers le vocabulaire.
✓	Le tokenizer convertit les IDs en texte lisible.

Environnement technique OmniASR

Stack modèle

- Python 3.11.9
- PyTorch 2.6.0 + CUDA 12.4
- fairseq2n 0.5.2
- omnilingual-asr 0.2.0

Audio & données

- torchaudio / soundfile
- FLAC 16 kHz
- pyarrow / Parquet
- format Hive-partitioned

Backend critique

- fairseq2n 0.5.2
- Compilation depuis sources
- CUDA_ARCHITECTURES="60;80"
- Support P100 + A100

HPC & exécution

- SLURM : sbatch / squeue / scancel
- helios : A100 40GB
- achille / heracles : P100 16GB
- caches locaux / TMPDIR

 **Points sensibles :** OOM GPU au dégel de l'encoder • erreurs NFS / quota disque • incompatibilités CUDA • mauvaise configuration SLURM

→ Avant le fine-tuning, l'étape critique a été de rendre OmniASR exécutable et stable sur le cluster.

Structure du projet OmniASR

```
MedBridge-AI2/  
├── cluster/  
│   ├── train_omnilingual_ctc.slurm  
│   └── eval_v6_clean.slurm  
├── configs/  
│   └── omnilingual_ctc_finetune_v6_clean.yaml  
├── custom_cards/  
│   └── med_custom_dataset.yaml  
├── scripts/  
│   ├── build_omni_v6.py  
│   ├── clean_omni_v6.py  
│   ├── audit_dataset.py  
│   └── omi/  
│       ├── create_omi_env.sh  
│       ├── setup_hpc.sh  
│       └── eval_checkpoint.py  
├── data/  
│   ├── parquet_omni_v6/  
│   └── parquet_omni_v6_clean/  
├── outputs/checkpoints/  
└── logs/
```

cluster/

- scripts SLURM
- training et évaluation GPU

configs/

- modèle, tokenizer, dataset
- batch, LR, checkpoints

custom_cards/

- déclaration du dataset
- lien avec fairseq2 assets

scripts/

- construction V6
- décontamination
- audit train/val/test

outputs/

- checkpoints, logs, métriques

→ Une structure claire garantit la reproductibilité, facilite le debug et évite les erreurs de configuration.

Fine-tuning OmniASR : Résultats

63.20%

WER brut

V6_clean · test

33.35%

WER normalisé

Éwé-spécifique

7

Versions

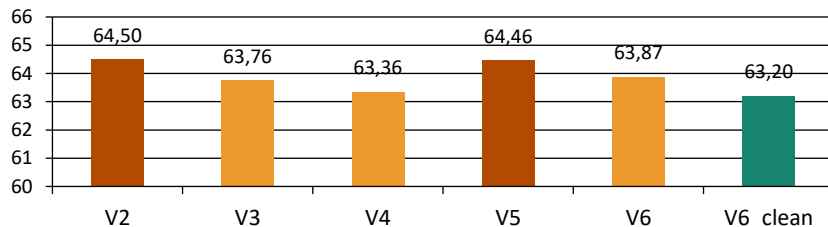
V1 → V6_clean

~150h

Données

Corpus Éwé

WER test brut par version (%)



Split	Nb	Durée (H)	Durée Min (S)	Durée Max (S)
train	2 4424	129,317	15,08	30,4
val	1 803	9,3139	14,18	29,56
test	1 611	8,6149	15,08	29,44

Lecture des résultats

Meilleur modèle **V6_clean, step 17 000**

WER val 34.91%

WER test brut **63.20%**

Écart apparent +28.3 points

L'écart val/test semblait suggérer un overfitting ou un shift de distribution. La normalisation montre que c'était principalement un artefact d'évaluation.

Découverte principale : biais d'évaluation

33.35%

WER réel après normalisation Éwé

vs 63.20% brut : le modèle capture la correspondance audio → texte, mais l'orthographe Éwé non normalisée pénalise le WER.

-30 pp

Écart brut → normalisé

Le WER brut surestime l'erreur réelle d'environ +89% relatif. L'écart val/test disparaît après normalisation.

Impact de la normalisation sur V6_clean

Niveau de normalisation	WER	CER
Brut (aucune)	63.20%	19.96%
Standard	34.06%	9.86%
Éwé-spécifique	33.35%	9.55%

Insight : CER brut $\approx 20\%$ << WER brut $\approx 63\%$ → les erreurs restantes sont majoritairement orthographiques (diacritiques, Unicode, espaces).

Confusions Éwé & reinterparation

ɛ → e | ɔ → o

ɖ ↔ d | ŋ ↔ n

ʋ ↔ v | ɣ ↔ g

+13 469 insertions d'espaces

Conclusion : WER val $\approx$ WER test après normalisation (~33–35%). Le problème principal n'est pas le modèle, mais la métrique brute appliquée à une orthographe Éwé hétérogène.

Taille du modèle : 1,21 Go

Synthèse & prochaines étapes

Résultat final

- WER brut : 63.20%
- WER normalisé : 33.35%
- CER normalisé : 9.55%
- Dataset : 27 833 échantillons

Conclusion scientifique

- Le WER brut surestime fortement l'erreur réelle.
- Le modèle apprend bien la correspondance acoustico-phonétique.
- Les erreurs résiduelles relèvent surtout de la normalisation orthographique.

Prochaines étapes

- grad_accum=1 avec batch réduit
- LR 5e-6 ou 3e-6
- Mixage de données (clean + noisy) / data augmentation
- Publication ICNLSP / Masakhane

Message clé : le système ASR est exploitable pour l'Éwé si l'évaluation et le post-traitement respectent l'orthographe et les diacritiques de la langue.



Contribution ASR low-resource

OmniASR CTC 300M sur l'Arabe égyptien

Hamad Aldenawi | Encadrement : Yannick Estève
Avignon Université · Master 1 APS · Mars–Avril 2026

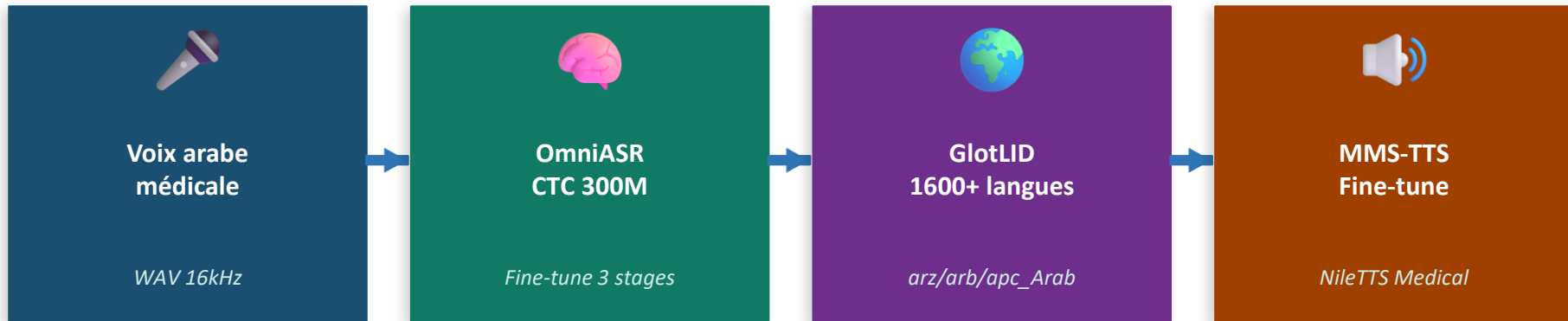
OmniASR 300M

GlottLID

MMS-TTS

WER 33.62%

Pipeline



Réalisation :

- Installation fairseq2n (compile depuis sources)
- Fine-tuning ASR en 3 stages (MGB3 -> NileTTS -> Medical)
- Pipeline ASR + LID complet
- Chunking audio pour audios longs (> 14s)
- Fine-tuning TTS sur NileTTS Medical
- Interface web Flask + démo Cloudflare

Fine-tuning ASR : Résultats

33.62%

WER Final

Stage 3 Medical

9.51%

CER Final

Char. Error Rate

3

Stages

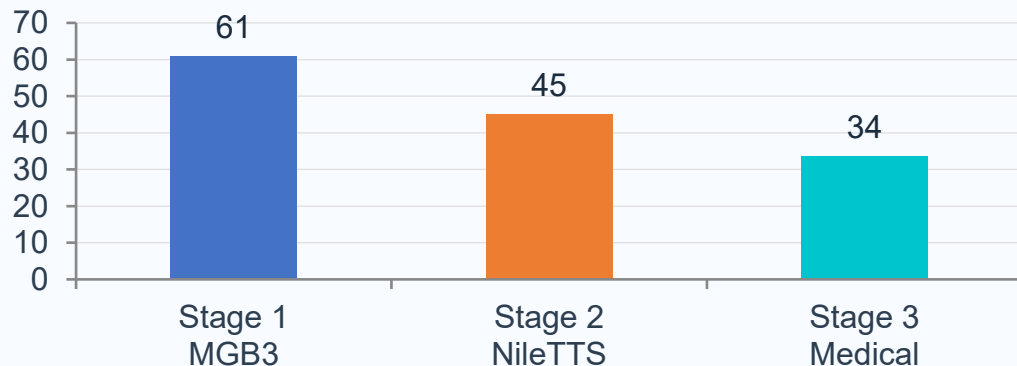
Curriculum Learning

38h

Données

NileTTS corpus

WER par stage (%)



Hyperparamètres :

Paramètre	Valeur
Optimizer	AdamW
LR stage 1/2/3	3e-6 / 5e-7 / 1e-7
Steps	3000 / 5000 / 3000
Batch eff.	8 (1 × grad_accum 8)
Precision	BFloat16
GPU	Tesla P100 16GB

Innovations techniques

Chunking audio (audios longs)

Problème : modèle limite à 14s

Solution : découpage en segments chevauchants



overlap 2s

→ fusion intelligente sans doublons

Identification de la langue : GlotLID

arb_Arab 99% • arz_Arab 68% • apc_Arab — 1600+ langues supportées

Fine-tuning TTS : MMS-TTS

Stage	Dataset	Loss
Stage 1	NileTTS 8571 utt.	~0.06
Stage 2	NileTTS Médical	0.0625

Mode 1: MMS-TTS fine-tune | Mode 2: Edge-TTS (fallback)

Résumé :



ASR

WER 33.62%

*CER 9.51% : 3 stages fine-tuning
OmniASR CTC 300M*



LID

~85-99%

*GlottID : 1600+ langues
Identification dialectes arabes*



TTS

Loss 0.0625

*MMS-TTS fine-tune medical
Edge-TTS fallback (ar-EG)*



Démo en direct : Interface web Flask (ASR + LID + TTS)

Données : NileTTS 38h | GPU : Tesla P100 / A100 | Déploiement : Cloudflare Tunnel

Hamad Aldenawi, APS Projet 6, Université d'Avignon, Avril 2026

Christ KELI

Reconnaissance Automatique de la Parole Éwé Fine-tuning Whisper Medium

Encadrement : Yannick Estève · Avignon Université · Master 2 APS · Mars–Avril 2026

WER Éwé 17.15%

−54% vs SOTA

Whisper Medium

30 642 exemples Éwé

Architecture Whisper : Transformer Seq2Seq

Qu'est-ce que Whisper ?

Modèle d'OpenAI pré-entraîné sur 680 000h d'audio multilingue. Architecture Transformer Seq2Seq (Encoder-Décodeur).

Encodeur

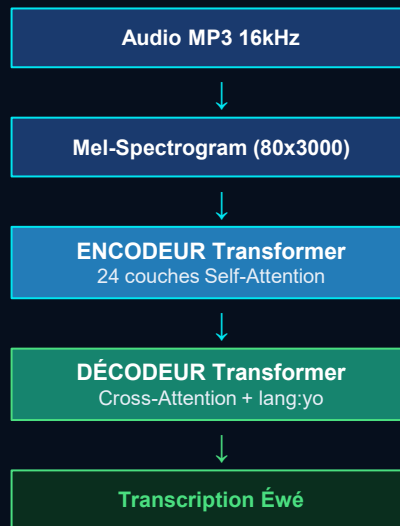
Transforme le mel-spectrogram (80 bandes x 3000 trames) en représentations contextuelles via 24 couches de Self-Attention. Capture les patterns acoustiques.

Décodeur

Génère la transcription token par token via Cross-Attention sur la sortie encodeur. Le token de langue (lang:yo) conditionne la génération.

Whisper Medium

307M paramètres · 24 couches · 1024 dimensions · 99 langues supportées. L'Éwé n'en fait pas partie → fine-tuning nécessaire.



Whisper Medium

307M param.
24 couches enc.
24 couches dec.
dim. 1024
99 langues

Fine-tuning

lr 1e-5
5 epochs
batch 16
warmup 200 steps
GPU A100 40GB

ASR Éwé : Résultats WER & CER

114.65%

WER Zero-shot

lang:yo · avant FT

17.15%

WER Fine-tuné

1443 ex. · pipeline exact

14.21%

WER Normalisé Éwé

mappings phonétiques Éwé

4.79%

CER Normalisé

Éwé-spécifique

Configuration	WER brut	CER brut	WER norm.	CER norm.
Zero-shot Whisper Medium (lang:yo)	114.65%	71.29%	111.39%	66.13%
Fine-tuné Yoruba ↗ (lang:yo)	17.15%	6.14%	14.21%	4.79%

Gain fine-tuning : WER brut 114.65% → 17.15% (−97 pts abs.) · SOTA Dei 2025 = 37% → dépassé de 54% relatif · Yoruba : langue tonale africaine, 2 phonèmes proches de l'Éwé (ɔ≈o, e≈e)

ASR Éwé : Problèmes & Solutions

✗ Problèmes rencontrés

Pipeline audio ffmpeg → WER 98%

Même modèle, pipeline différent → features incompatibles avec l'entraînement. WER 27% → 98% selon le décodeur MP3.

Tokenizer anglais → caractères Éwé absents

lang:en : combinaisons approximatives de tokens → convergence bloquée à WER 100% pendant 3+ epochs.

ForceTokensLogitsProcessor conflit

forced_decoder_ids dans model.config ET dans generate() → ValueError.
ZeroDivisionError : preds vides silencieusement.

WER 99% fine-tuné sans lang:yo forcé

Évaluation sans forced_decoder_ids → modèle génère n'importe quelle langue. WER 99% au lieu de 17%.

✓ Solutions appliquées

librosa.load(BytesIO) — pipeline exact

Même décodeur MP3 à l'entraînement et à l'inférence. WER remonté à 17.15%.

lang:yo (Yoruba) comme décodeur

Langue tonale africaine, 2 phonèmes proches Éwé. WER 63% dès epoch 0.05 vs 100% avec lang:en.

model.config.forced_decoder_ids = None

Vider config avant generate(), mettre forced_ids dans model.config uniquement. Élimine le conflit double.

Forcer lang:yo à l'inférence aussi

Conditions entraînement = conditions évaluation. WER 99% → 17.15% en appliquant ce correctif.

Leçon principale : Le pipeline audio est aussi critique que les hyperparamètres · La langue de décodage doit être identique à l'entraînement et à l'inférence · Yoruba = meilleure approximation disponible dans Whisper pour une langue tonale africaine proche de l'Éwé

Synthèse Éwé Toutes métriques WER/CER comparées

Configuration	WER brut	CER brut	WER normé	CER normé	Gain
Zero-shot Whisper (lang:en)	~210%	~85%	~180%	~70%	—
Zero-shot Whisper (lang:yo)	114.65%	71.29%	111.39%	66.13%	—
Fine-tuné V2 (lang:yo moitié des données)	27.42%	—	—	—	-9.58
Fine-tuné Yoruba ↗ (lang:yo)	17.15%	6.14%	14.21%	4.79%	-54%

Référence SOTA : Dei et al. ICNLSP 2025 — Whisper Base sur Éwé — WER 37% · Gain relatif : $(37-17.15)/37 = 53.6\% \approx -54\%$

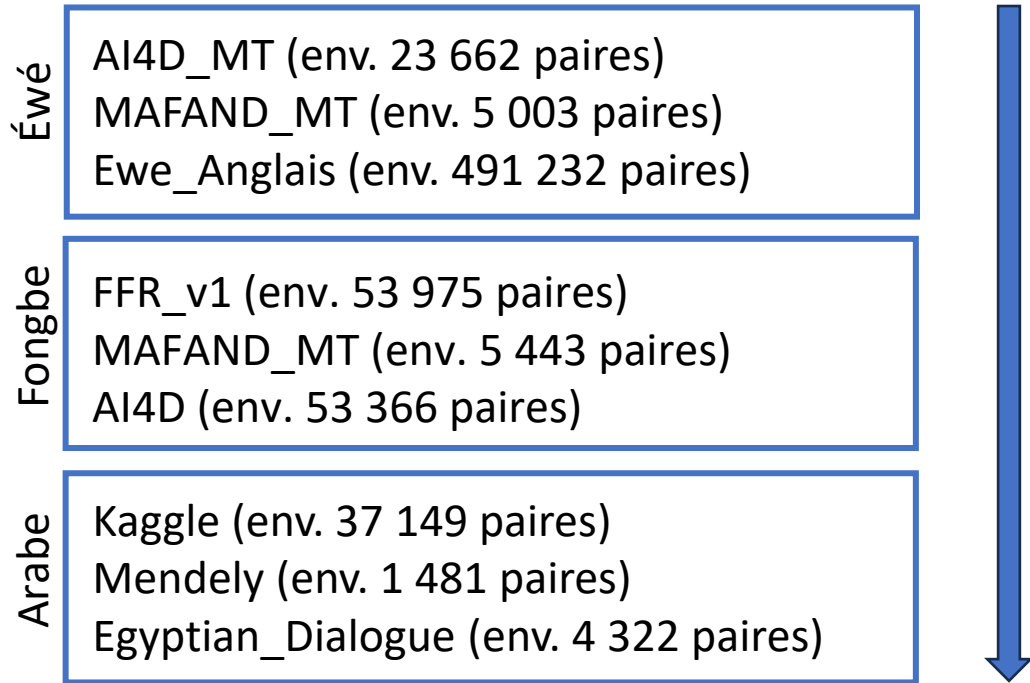
Bilan : La normalisation Éwé révèle que le WER réel est 14.21% · Le CER normalisé 4.79% confirme que le modèle capture bien les phonèmes Éwé · Pipeline librosa BytesIO = condition critique pour reproduire ces résultats

Contribution MT low-resource

NLLB-200-1.3B

*Patrice SEBASTIANO | Encadrement : Yannick Estève
Avignon Université · Master 1 APS · Mars–Avril 2026*

Données : téléchargement et prétraitement



Download

réalisé avec des scripts individuels par Dataset -> portabilité et répétitivité

Prétraitement

Idem : un script par dataset pour uniformiser dans une structure unique

Database_origine | ID | text_src | text_fr

Analyse

Kaggle -> Fuite sémantique

AI4D -> Données répétitives



Éwé - Fine tuning et Résultats

Quatre expériences réalisées

Époque	Config 1 (Baseline) Val		Config 2 (+ Fongbe) Val		Config 3 (Synthétique 1:2) Val		Config 4 (Synthétique 1:4) Val	
	Validation	Test	Validation	Test	Validation	Test	Validation	Test
0 (initial)	4,62	27,1	4,62	27,1	4,62	27,1	4,62	27,1
1	15,29		12,52		13,36		16,68	
2	15,82		12,02		15,88		13,3	
3	16,17		16,35		12,42		12,95	24,92
4	16,34		23,33		16,62			
5	16,05		17,57		13,17			
6 (final)	16,34	18,21	16,33	11,85	13,4	16,82		

Analyse

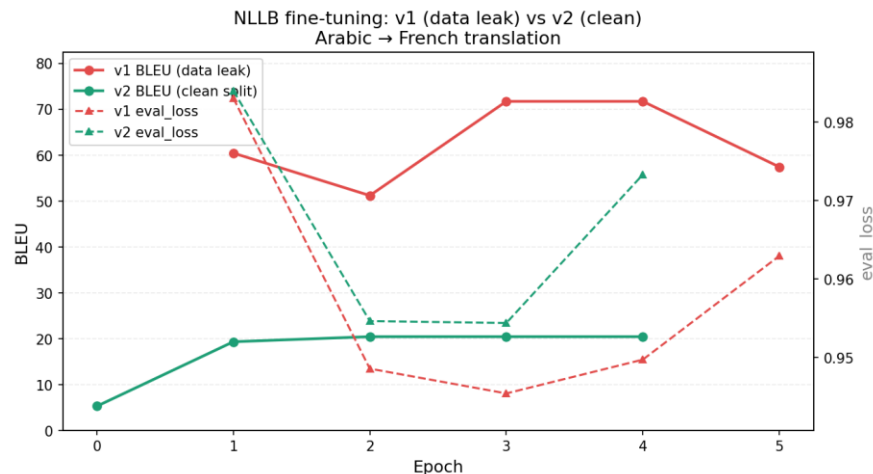
- Introduire le Fongbe n'a pas aidé le modèle
- L'expérience 1 reste la plus stable
- L'expérience 4 montre que les données affectent entraînement

Configuration	BLEU Val	BLEU Test	Moyenne
One-shot (sans fine-tuning)	4,62	27,1	15,86
Config. 1 — Baseline	16,34	18,21	17,28
Config. 3 — Synthétique 1 :2	13,4	16,82	15,11
Config. 4 — Synthétique 1 :4	12,95	24,92	18,93

Arabe égyptien - Fine tuning et Résultats

Deux expériences réalisées

Époque	Expérience v1 - BLEU		Expérience v2 - BLEU	
	BLEU	Loss	BLEU	Loss
0 (initial)	5,3	1,575	5,3	1,575
1	60,43	0,983	19,34	0,984
2	51,18	0,949	20,45	0,955
3	71,69	0,95	20,45	0,954
4 (final)	71,69	0,963	20,45	0,973
5 (stop v1)	57,42			



Analyse

- **v1 :** Eval loss stable $\neq$ BLEU fiable $\rightarrow$ la métrique est corrompue par la fuite sémantique
BLEU instable = Mémorisation variable de structures contaminées selon l'époque
- **v2 :** sans fuite : loss et BLEU convergent ensemble $\rightarrow$ signal d'entraînement cohérent



Triangulation sémantique

Constat : Les expériences précédentes ont permis d'améliorer les traductions
Éwé ↔ Français et Arabe égyptien ↔ Français.

Problème : Comment aligner alors les datasets pour réaliser une fine tuning Éwé ↔ Français et Arabe égyptien

Le tableau montre ce qui est obtenu en fusionnant les deux datasets sur la colonne Français : sur une ligne, une information est forcément manquante.

Éwé	Français	Arabe égyptien
X	X	-
-	X	X
X	X	-
X	X	-
-	X	X
X	X	-

Triangulation sémantique

Modèle : paraphrase-multilingual-mpnet-base-v2: ce dernier génère des embeddings agnostiques à la langue

Idée : Utiliser les relations entre embeddings connus (français) pour trianguler la position des textes manquants (soit Arabe égyptien, soit Éwé)

Algorithme

① Embedding

Encoder toutes les paires $\rightarrow$ fr_embeddings + ar/ewe_embeddings

② Distance matrix

Calculer fr_dist_matrix — distances cosinus entre tous les vecteurs français

③ Partition

70% masked (cibles cachées) · 30% known (ancres de référence)

④ Prédiction par voisinage (pour chaque paire masquée)

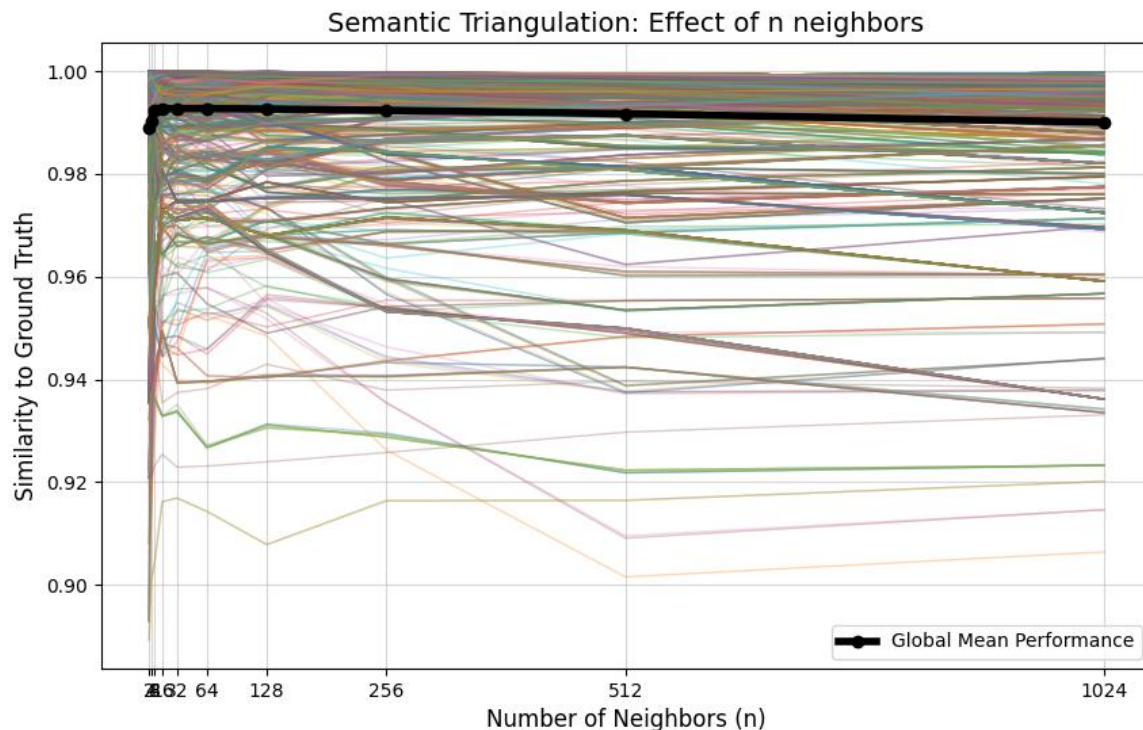
- a. Trouver les n voisins français les plus proches dans known
- b. $\text{embed_shift} = \text{fr}[\text{target}] - \text{fr}[\text{neighbor}] \rightarrow \text{pred} = \text{ar}[\text{neighbor}] + \text{embed_shift}$
- c. Moyenner les n prédictions $\rightarrow$ final_prediction

⑤ Évaluation

$\text{truth_sim} = \text{cosine_similarity}(\text{final_prediction}, \text{ar}[\text{target}]) \cdot n \in [2 \dots 1024]$

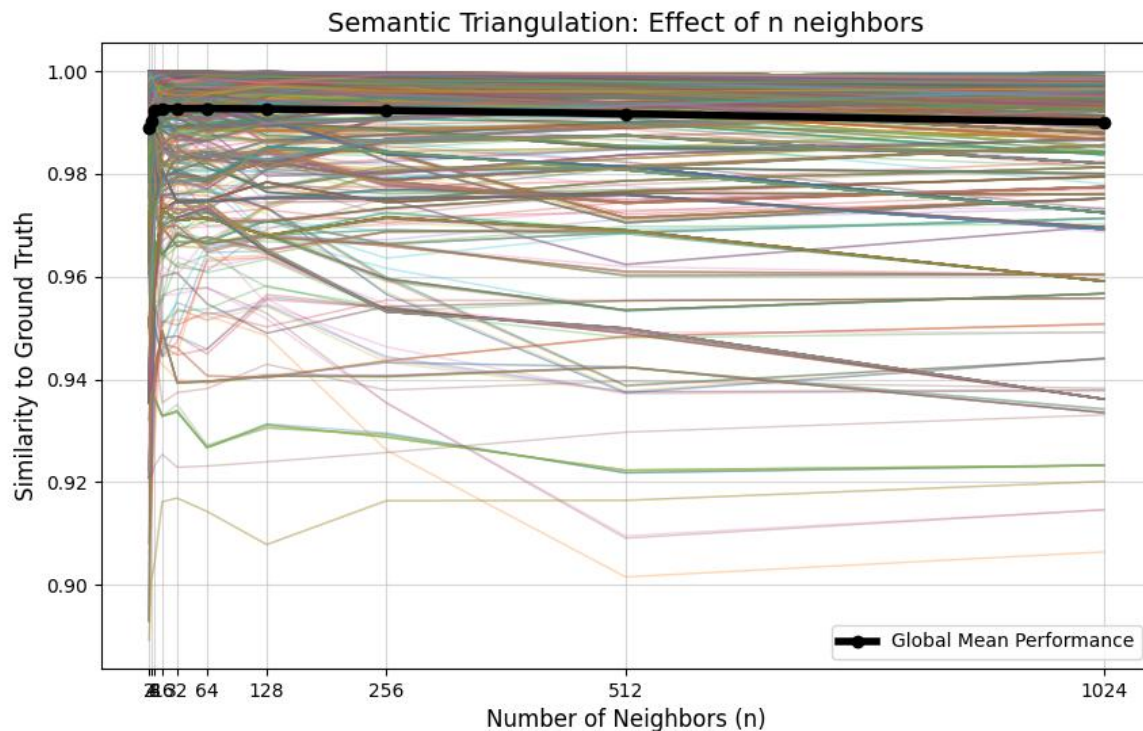
Triangulation sémantique

Résultats Arabe égyptien – Précision moyenne (4000 paires, 70% masquée) : 0,9927 %



Triangulation sémantique

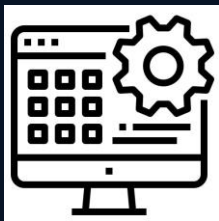
Résultats Éwé - Précision moyenne (4000 paires, 70% masquée) : 0,9978 %



NLLB-200-1.3B

Principales Conclusions

- Pour l'Éwé : BLEU de 15,86 à 18,93 -> + 20 %
- Pour l'Arabe égyptien : BLEU de 5,30 à 20,45 -> + 385 %
- Alignement des deux datasets :
 - La triangulation sémantique avec le modèle mpbasev2 est extrêmement précise
 - Il reste le problème du décodage des états cachés vers le texte.



Démo

Avignon Université · Master 1 APS · Mars–Avril 2026

OmniASR Éwé

OmniASR Arabe

Wishper Éwé

MT (Éwé / Arabe) →
(Français / Anglais)



MedBridge AI

Annexe

Accédez à nos dépôts et ressources

Scannez les QR codes ou cliquez sur les liens ci-dessous pour accéder à nos projets et modèles.



GitHub



Code source, scripts d'entraînement, pipeline et outils du projet MedBridge AI.



Accéder au dépôt

<https://github.com/steve-sanogo/medbridge-speech-stack>



Hugging Face



Modèles pré-entraînés, datasets et ressources associées du projet MedBridge AI.



Accéder au dépôt

<https://huggingface.co/medbridge-ai>



Merci de consulter nos dépôts pour plus de détails, reproduire nos résultats et contribuer au projet !